

Christof Teuscher

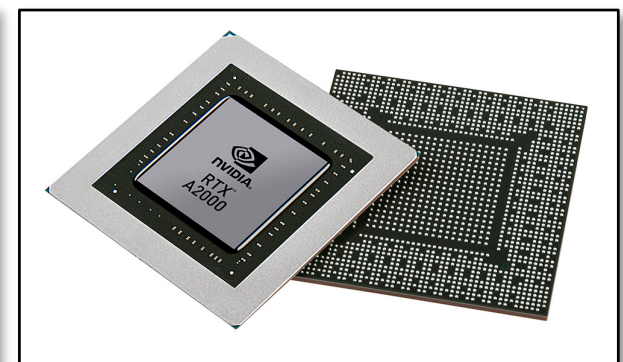
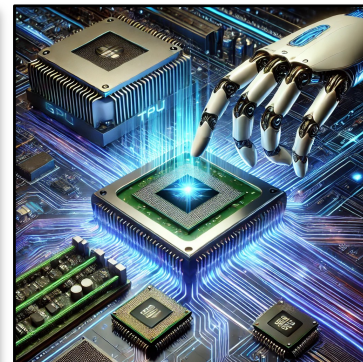
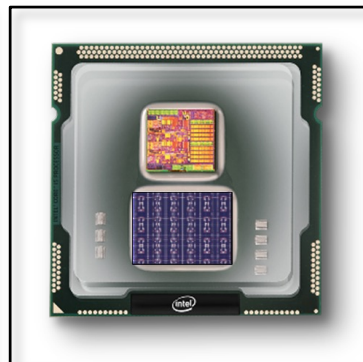
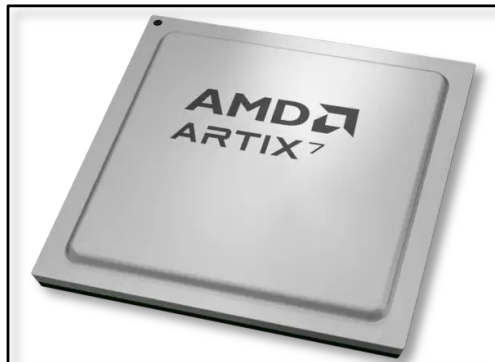
ECE 410/510: Hardware for AI and ML, Spring 2026

Hardware for AI and ML: an Overview

Portland State University
Department of Electrical and Computer Engineering (ECE)

www.teuscher-lab.com

teuscher@pdx.edu





Christof Teuscher



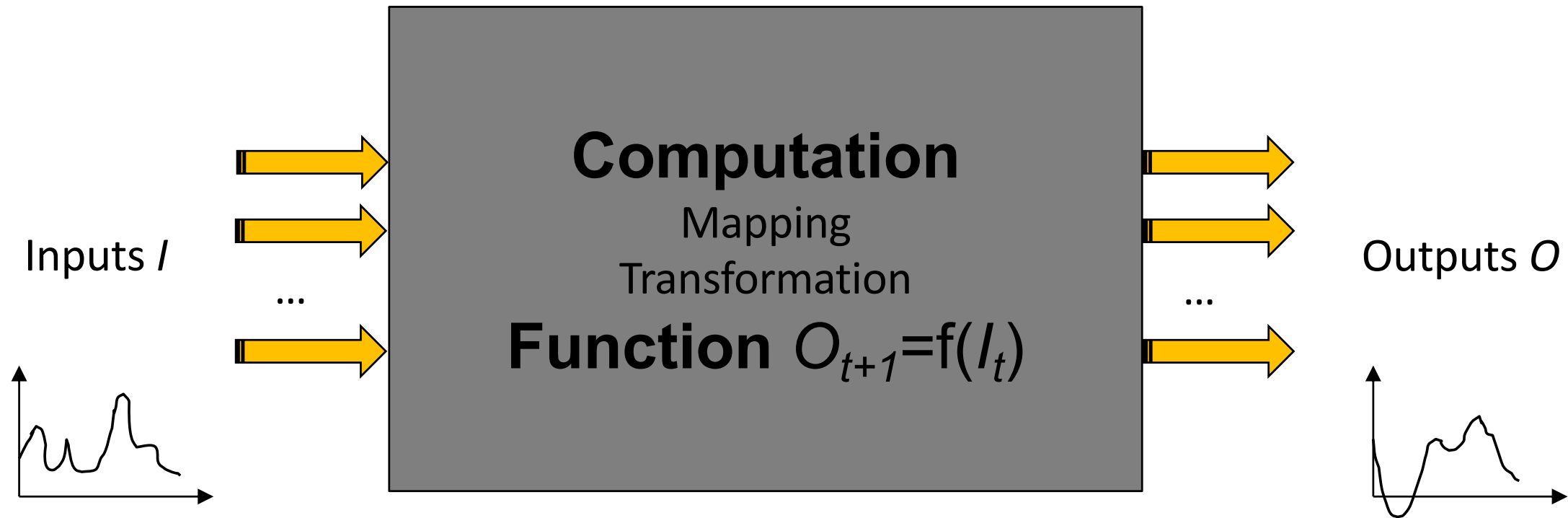
teuscher@pdx.edu

www.teuscher-lab.com/teaching



What is a computer?

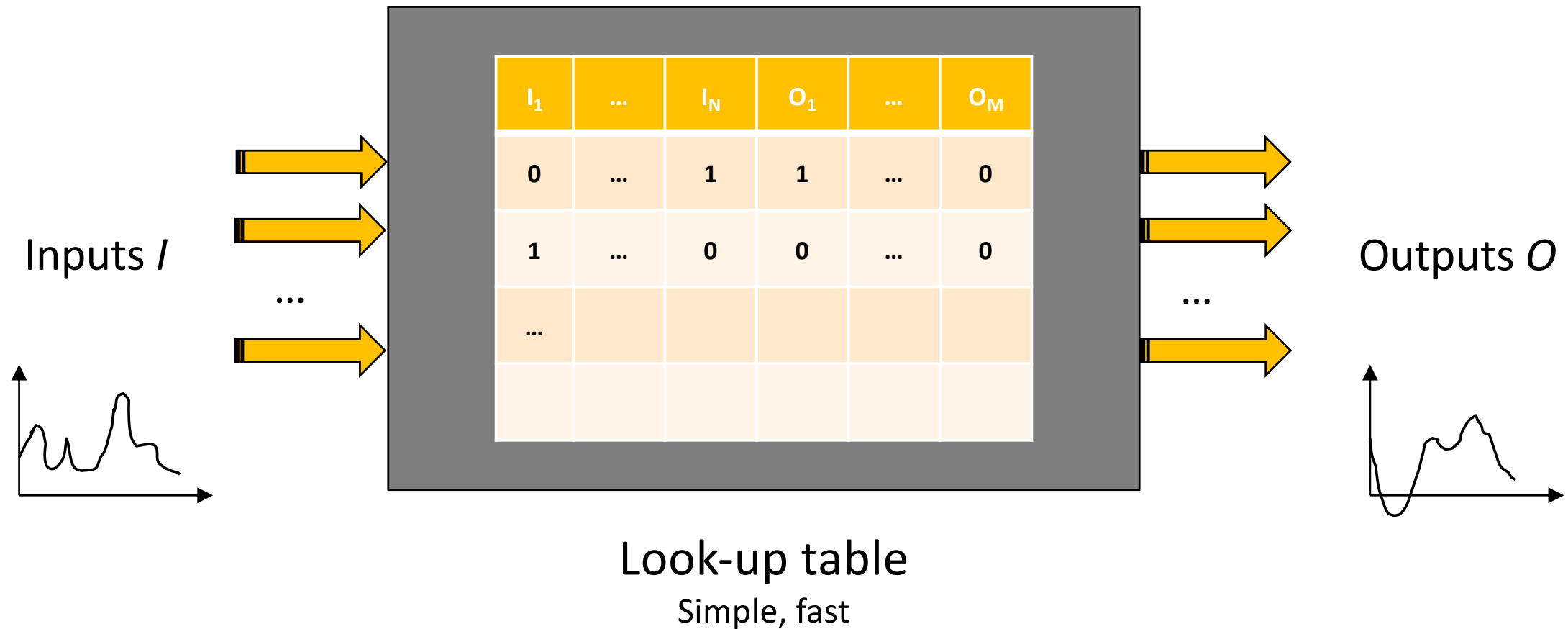
What is computation? (1)



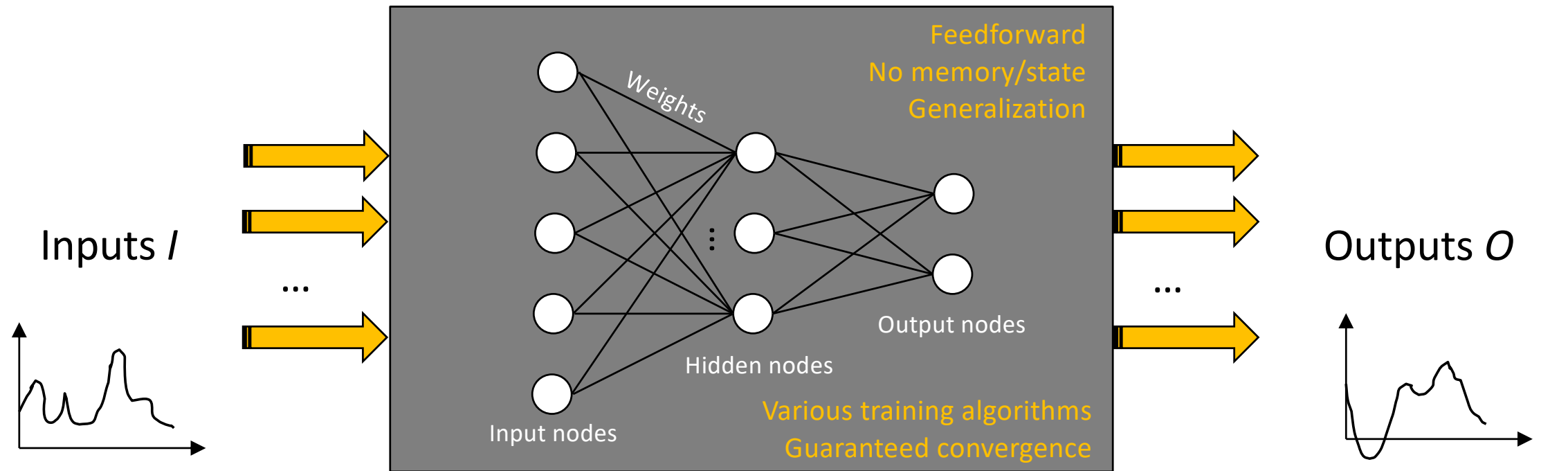
How can we “realize” this mapping?



What is computation? (2)



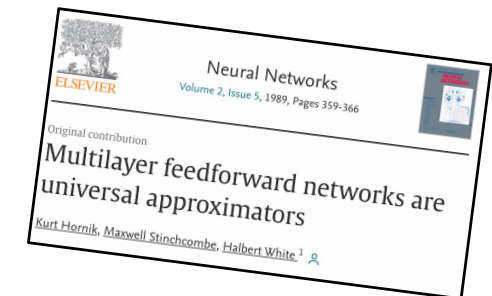
What is computation? (3)



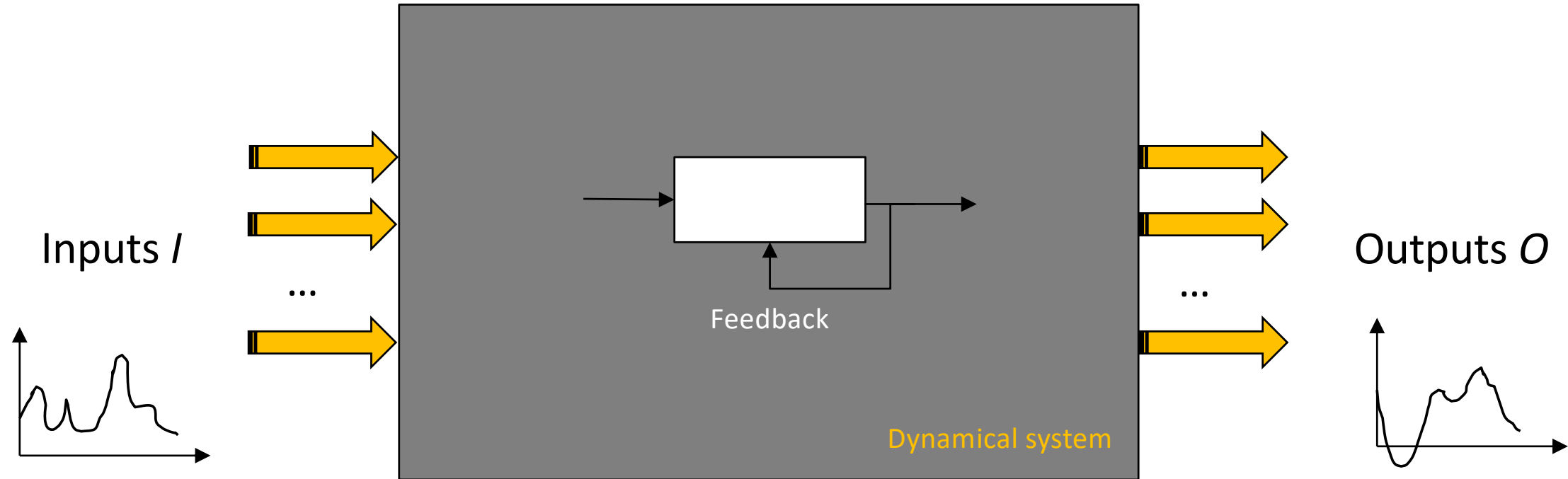
Neural network

Universal function approximator

(Hornik, Stinchcombe, White, 1989, [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8))



What is computation? (4)



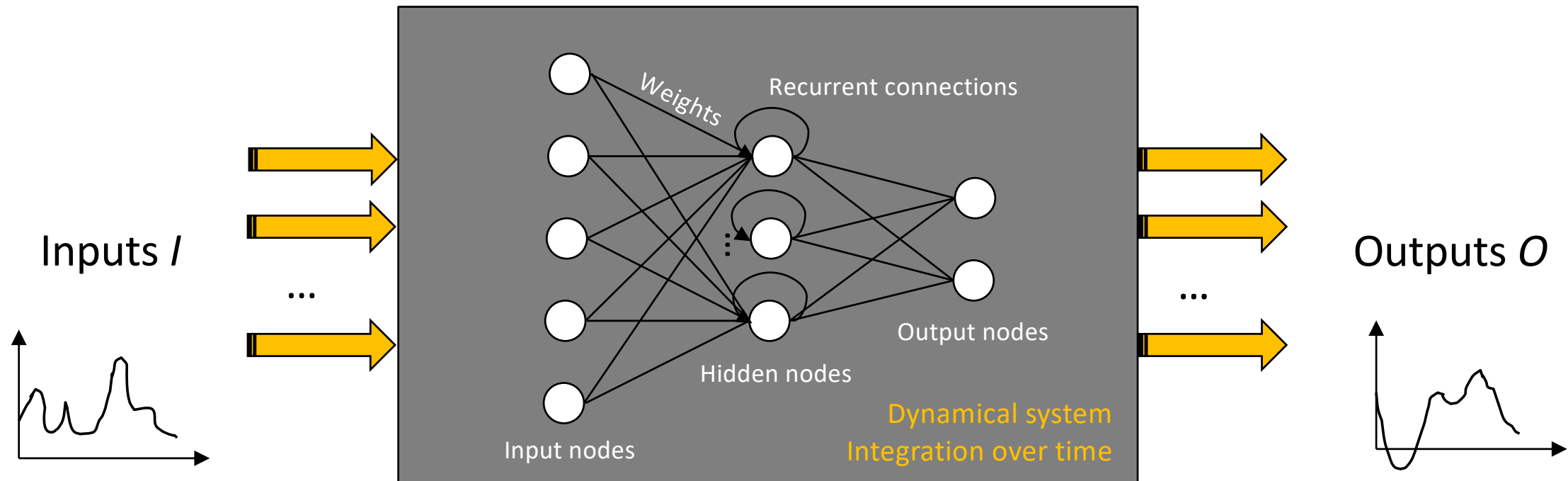
Memory is needed in a lot of cases

Temporal signal processing (audio, video)

Timeseries prediction

Control tasks

What is computation? (5)



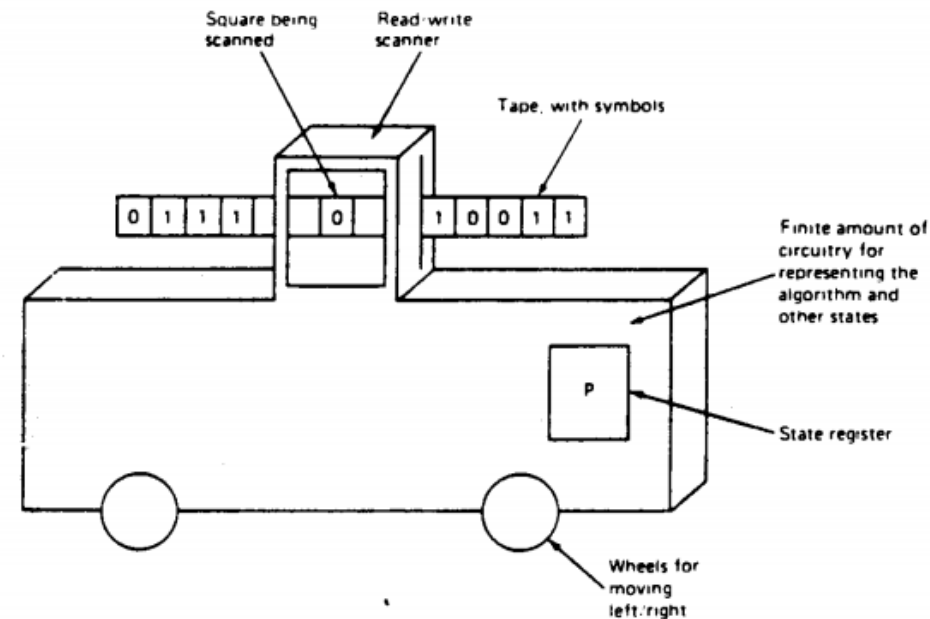
Recurrent neural networks (RNN)

Training is challenging, various algorithms exist (backpropagation through time), convergence not guaranteed, many weights, slow learning. RNNs are also universal function approximators

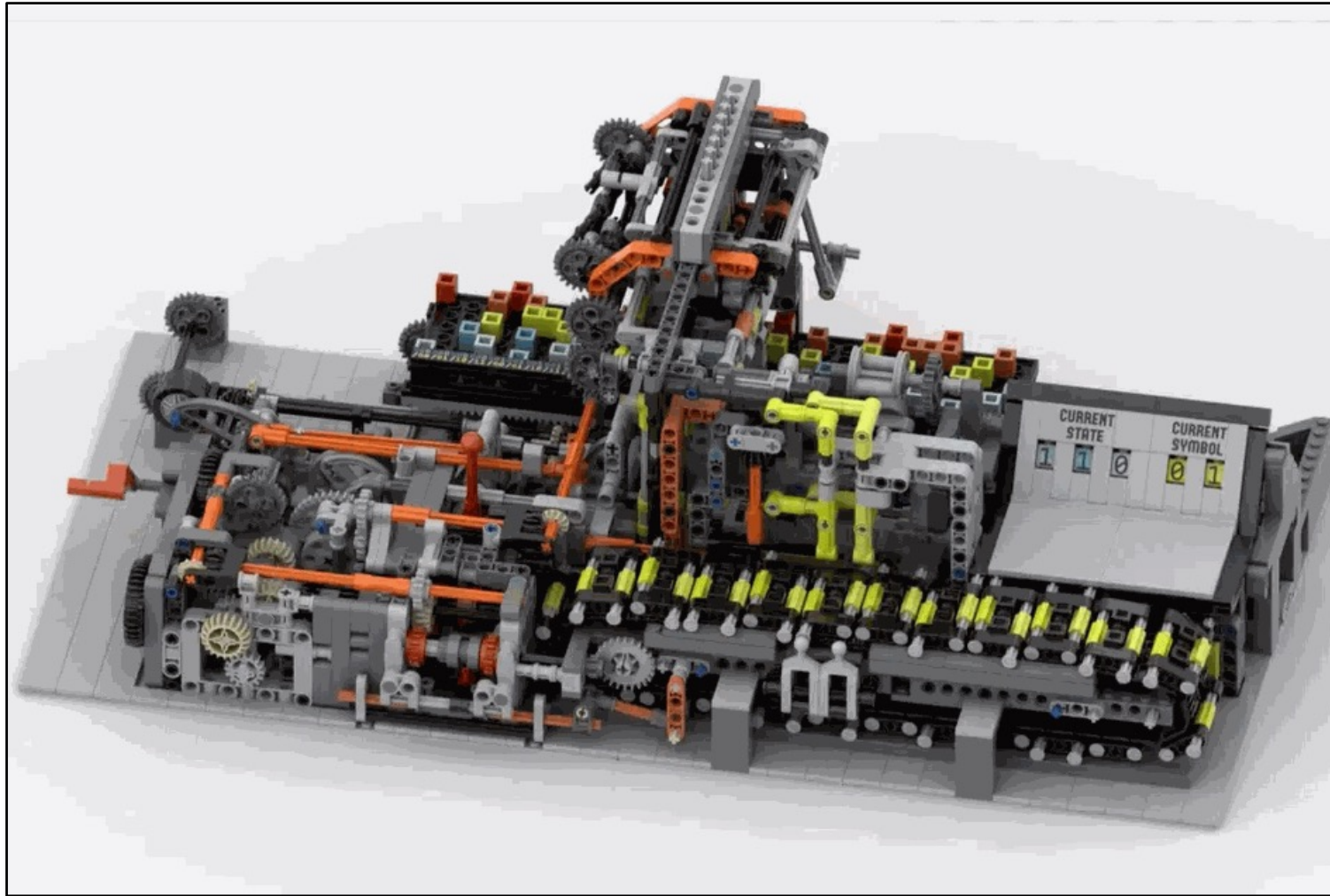
(Schaefer & Zimmerman, 2007, <https://doi.org/10.1142/S0129065707001111>)

Turing Machine (TM)

Figure B-2 An imaginary, physical Turing machine.



- Turing's aim was to define the simplest possible device that could perform any computation that could be performed by a human.
- TMs are a model of computability, they do not model the physical process of a computer.



Turing Machine (TM)

“What does a Turing machine look like? You can certainly imagine some crazy looking machine, but a better approach is to look in a mirror.”

— Charles Petzold, The Annotated Turing

<https://ideas.lego.com/projects/10a3239f-4562-4d23-ba8e-f4fc94eef5c7>

Intrinsic vs designed computation (1)

- **Designed computation:**

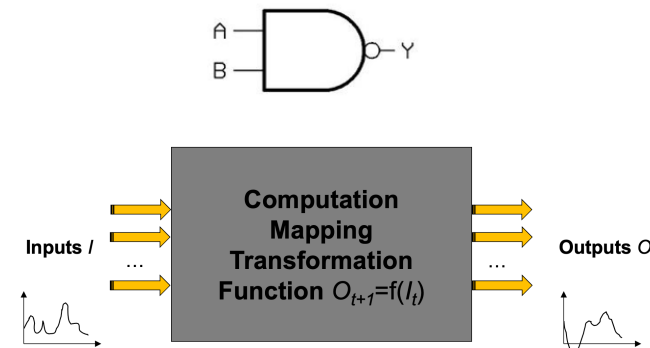
- top-down
- we design devices and circuits for our purpose
- information processing that is “useful”

- **Intrinsic computation:**

- inherent dynamics of a dynamical system
- not necessarily “useful”

- **Everything computes something, the question is what and how it can be harnessed for “useful” purposes.**

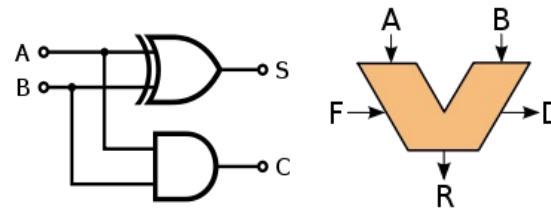
- *“What natural substrates are complex enough to support controllable and useful information processing?”* (Crutchfield et al., 2010, <https://doi.org/10.1063/1.3492712>)



What does a falling apple “compute?”



Intrinsic vs designed computation (2)



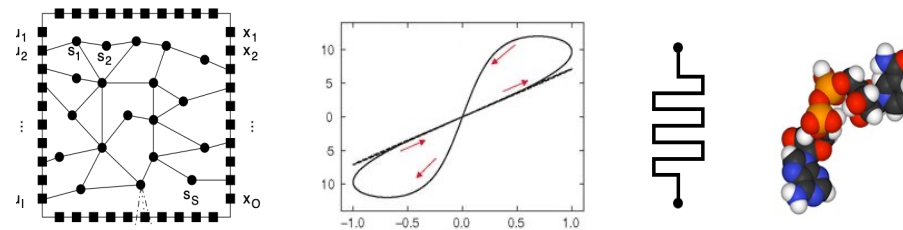
Desired computation

Computing architecture

Intrinsic dynamics

Device network

How to obtain a desired computation?



Intrinsic vs designed computation (3)

“Evolvable hardware” variant

Nanocell Logic Gates for Molecular Computing

James M. Tour, William L. Van Zandt, Christopher P. Husband, Summer M. Husband, Lauren S. Wilson, Paul D. Franzon, and David P. Nackashi

(Tour et al., 2002, <http://doi.org/10.1109/TNANO.2002.804744>)

- Network trained by switching NDR switches ON/OFF at post fabrication.
- Omnipotence is assumed (one knows and can control all switches)
- Tasks: logic functions, 1 bit-adder
- In simulation only

Self-assembled network of metallic particles, with reprogrammable negative differential resistance (NDR)

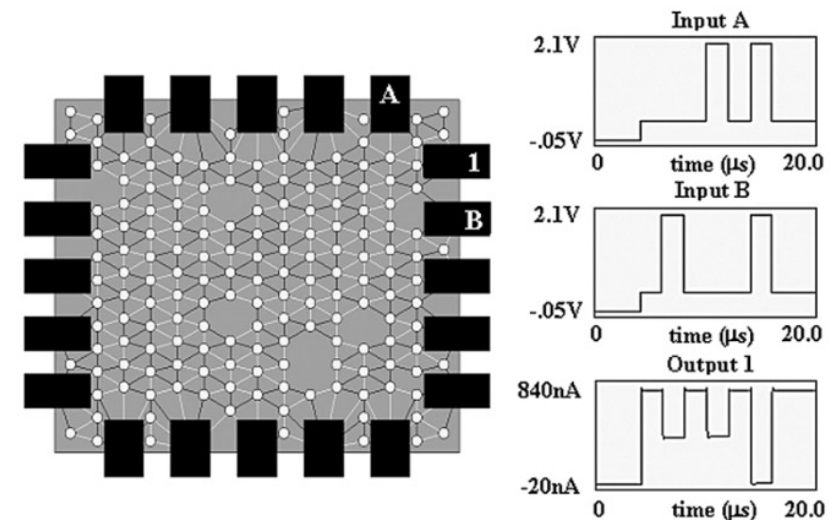


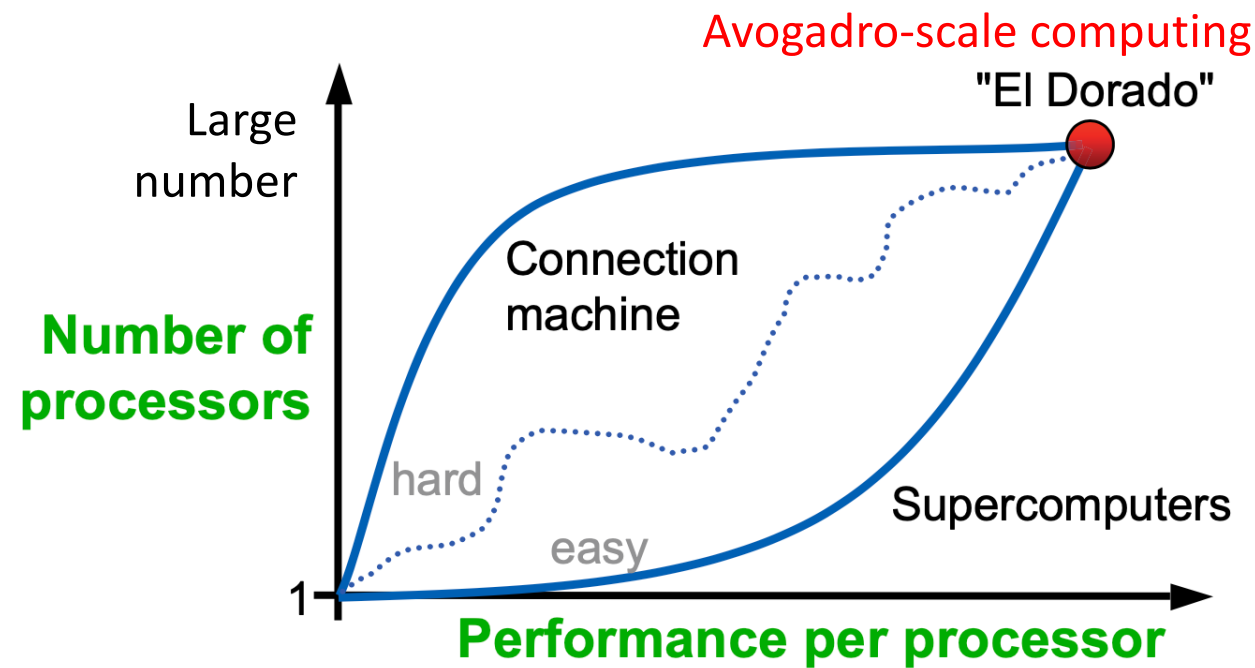
Fig. 5. NAND gate is demonstrated in this nanocell using the SPICE interface model and the $I(V)$ curve shown in Fig. 3. The plots show the $V(t)$ and $I(t)$ with the accompanying NAND Truth Table logic.



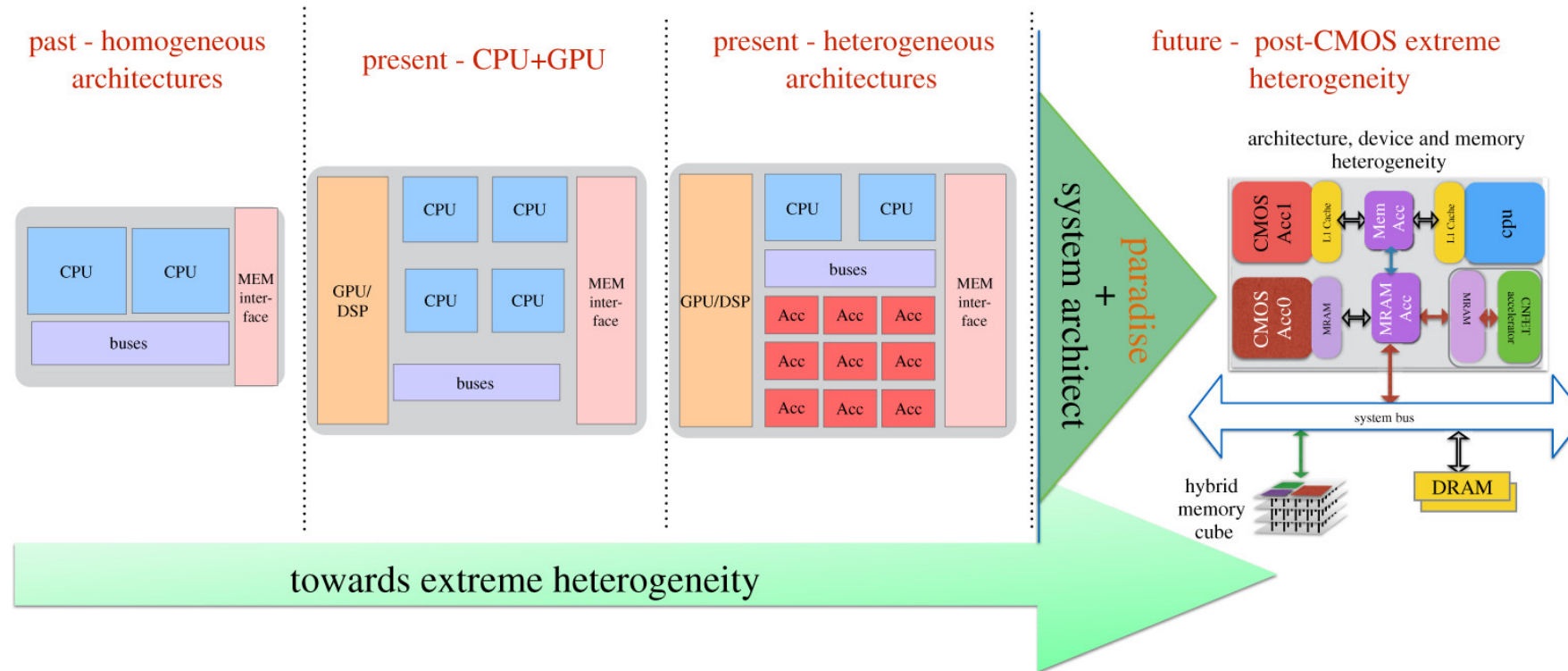
What kind of computing machinery would an extraterrestrial build?



The goals and drivers have changed



The goals and drivers have changed

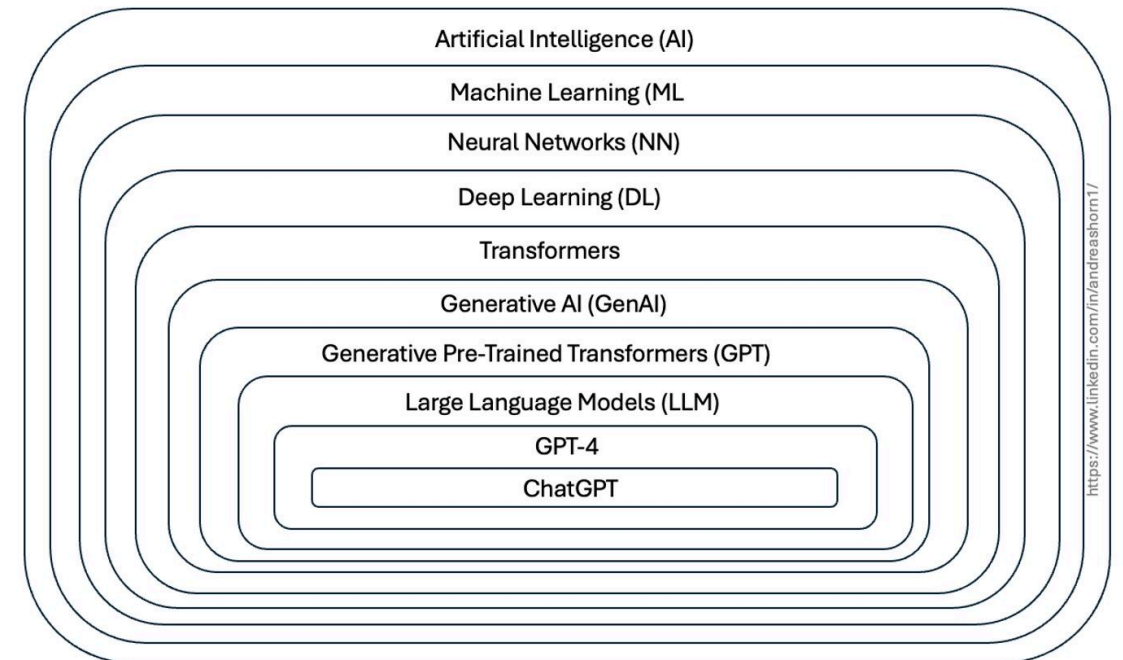
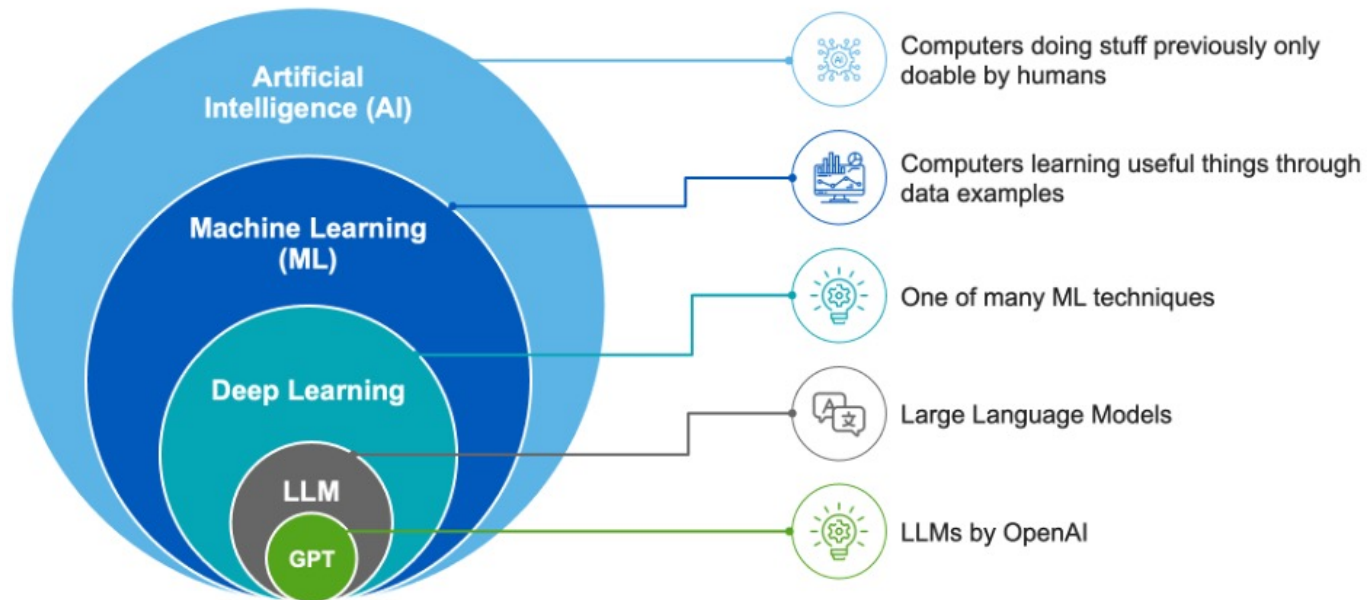


<https://royalsocietypublishing.org/doi/10.1098/rsta.2019.0061>

AI vs ML

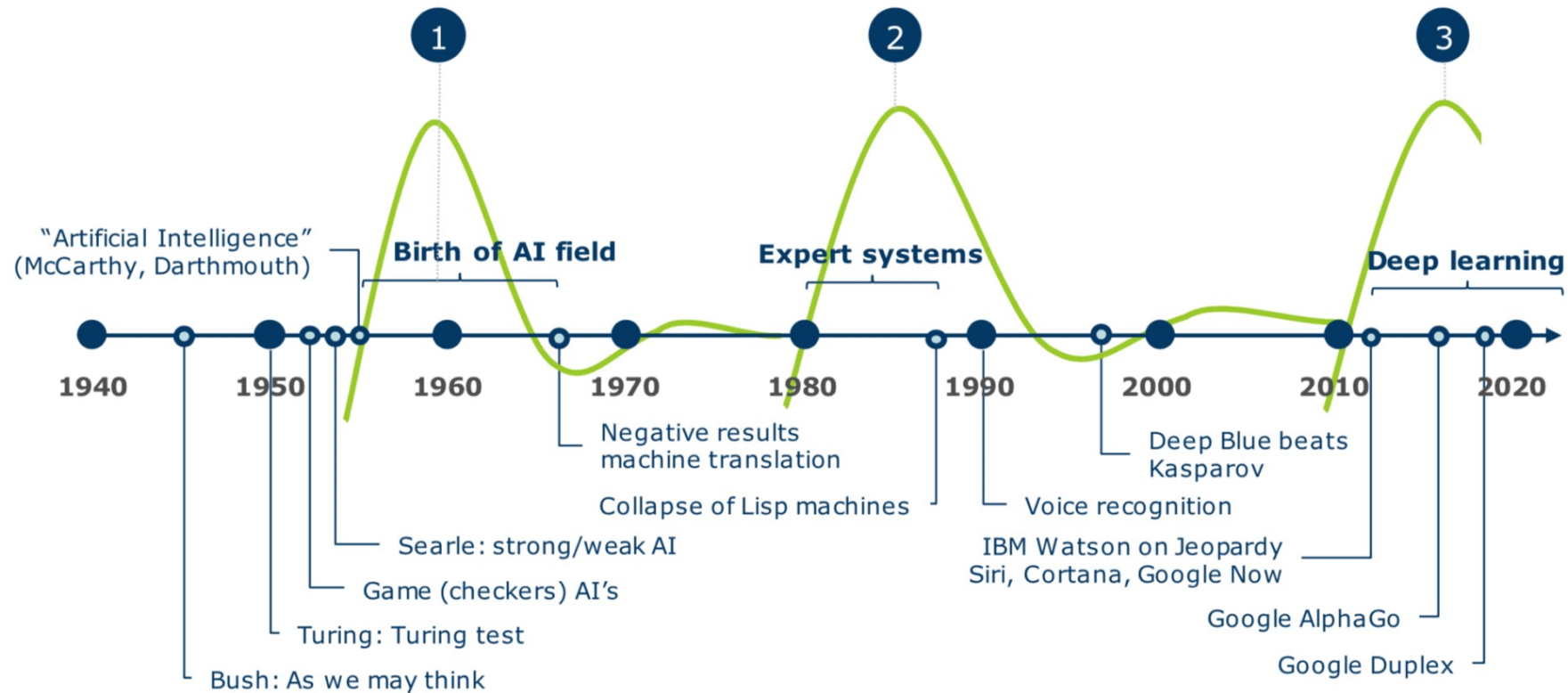
- **Machine Learning (ML)** is one technique used to achieve **Artificial Intelligence (AI)**.
- All ML is AI, but not all AI is ML.
- AI is the destination (e.g., intelligent machines), while ML is one path to get there (learning from data).
- **Artificial Intelligence (AI)**: Broader concept encompassing machines that can perform tasks requiring human intelligence. Ultimate goal: Artificial General Intelligence (AGI).
- **Machine Learning (ML)**: Subset of AI focused specifically on algorithms that improve through experience. Example: neural networks.

AI vs ML



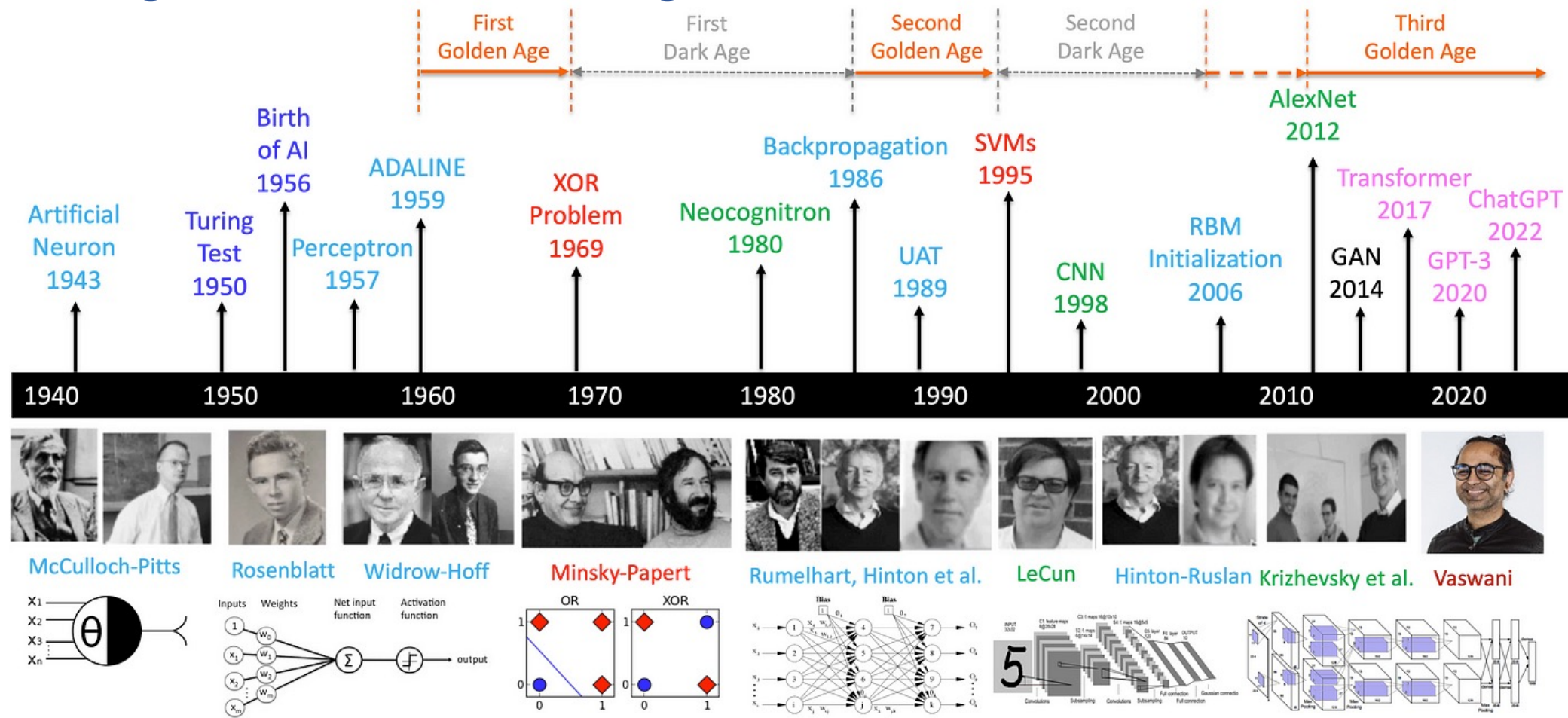
Source: Tignis

A very brief history of AI



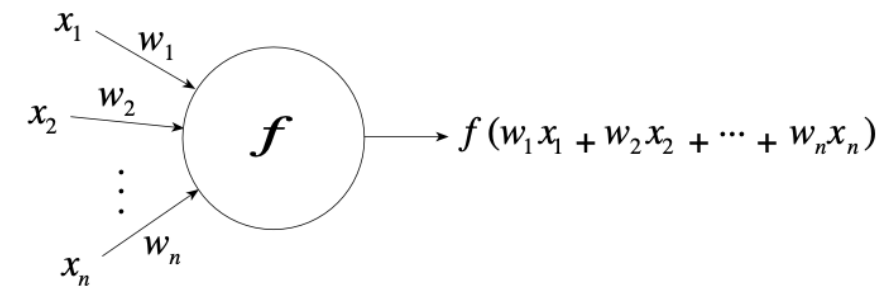
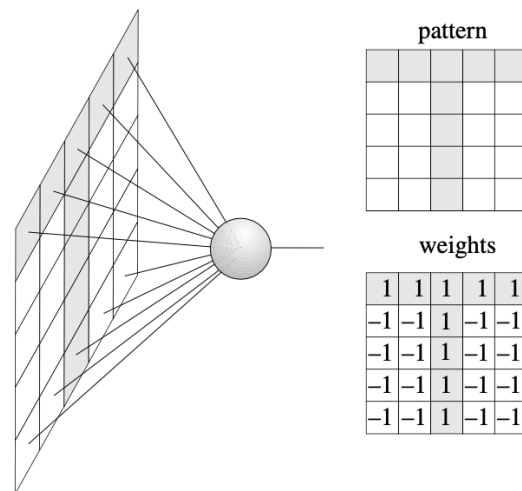
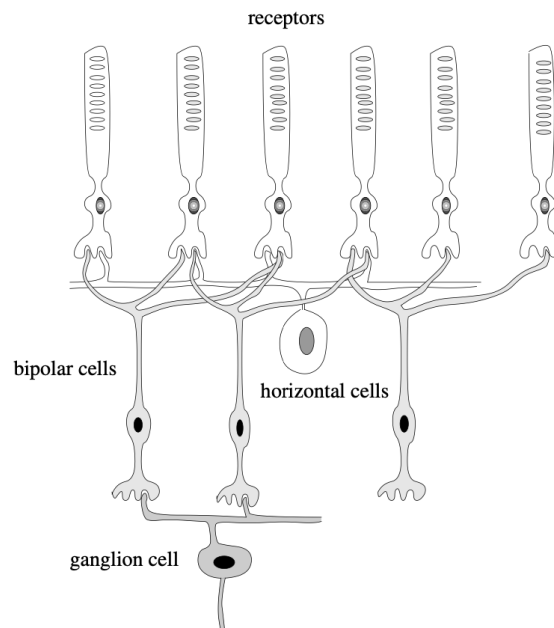
Strong AI = AGI = Artificial General Intelligence

A very brief history of AI with deep learning

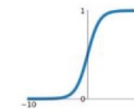


<https://medium.com/@lmpo/a-brief-history-of-ai-with-deep-learning-26f7948bc87b>

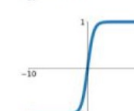
An abstract neuron



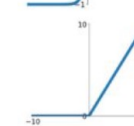
Sigmoid
 $\sigma(x) = \frac{1}{1+e^{-x}}$



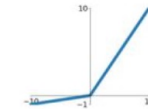
tanh
 $\tanh(x)$



ReLU
 $\max(0, x)$



Leaky ReLU
 $\max(0.1x, x)$



Maxout
 $\max(w_1^T x + b_1, w_2^T x + b_2)$

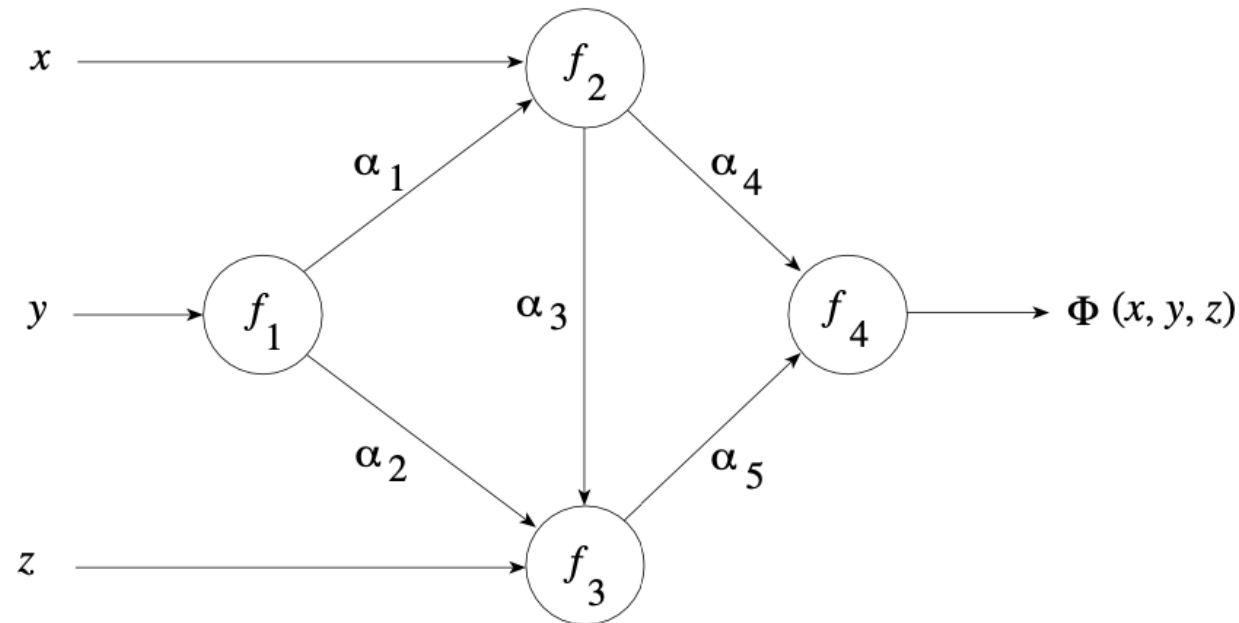
ELU
 $\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$



Rojas, 1986



A neural network



Rojas, 1986

Special types of neural networks

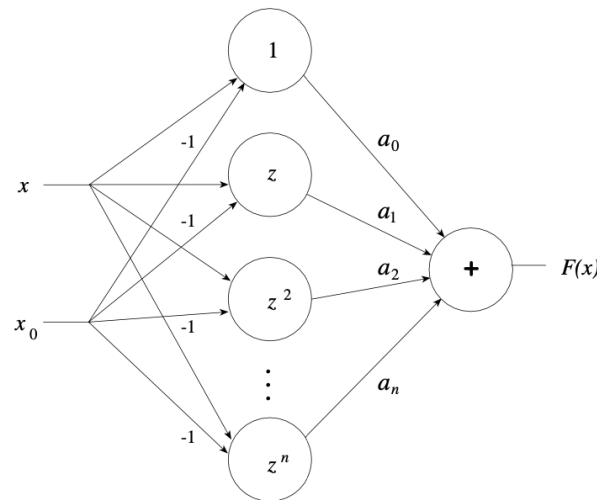


Fig. 1.16. A Taylor network

$$F(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)^2 + \cdots + a_n(x - x_0)^n + \cdots,$$

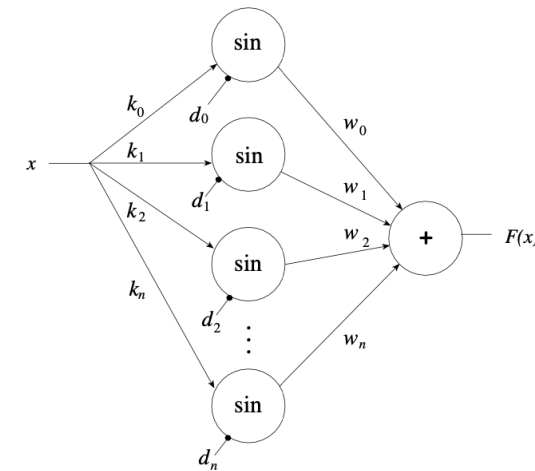


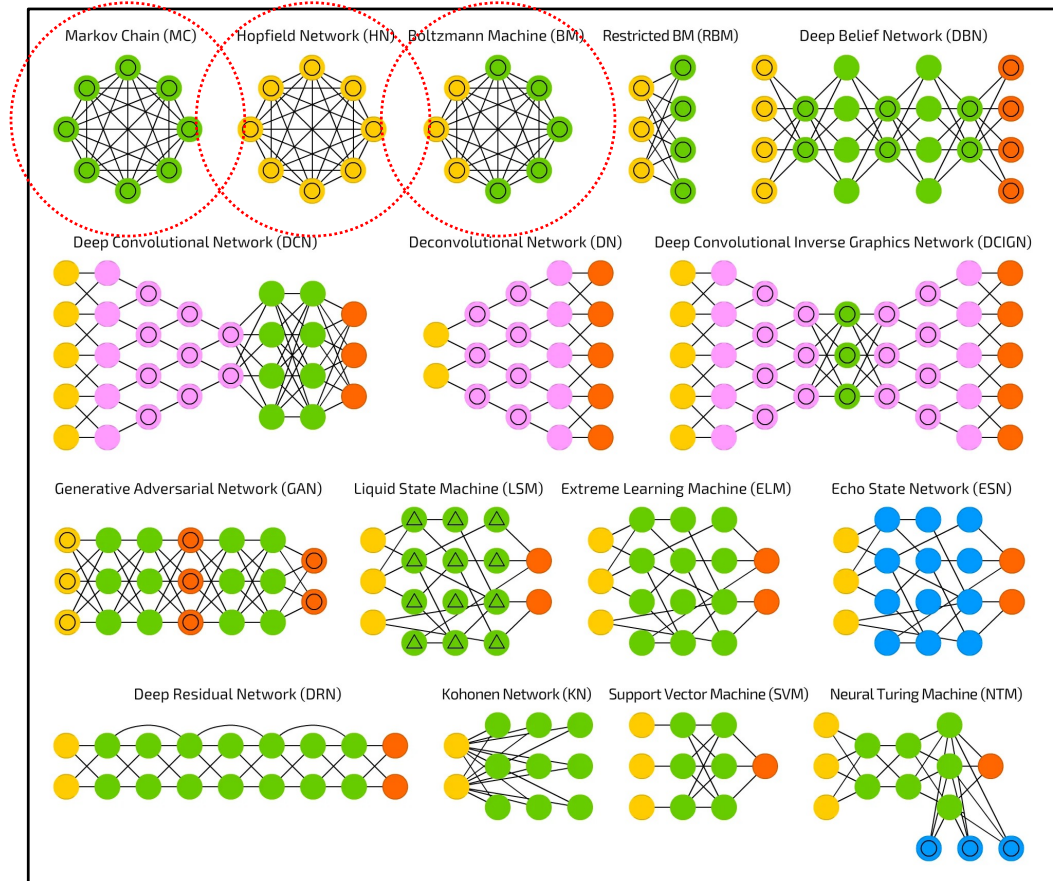
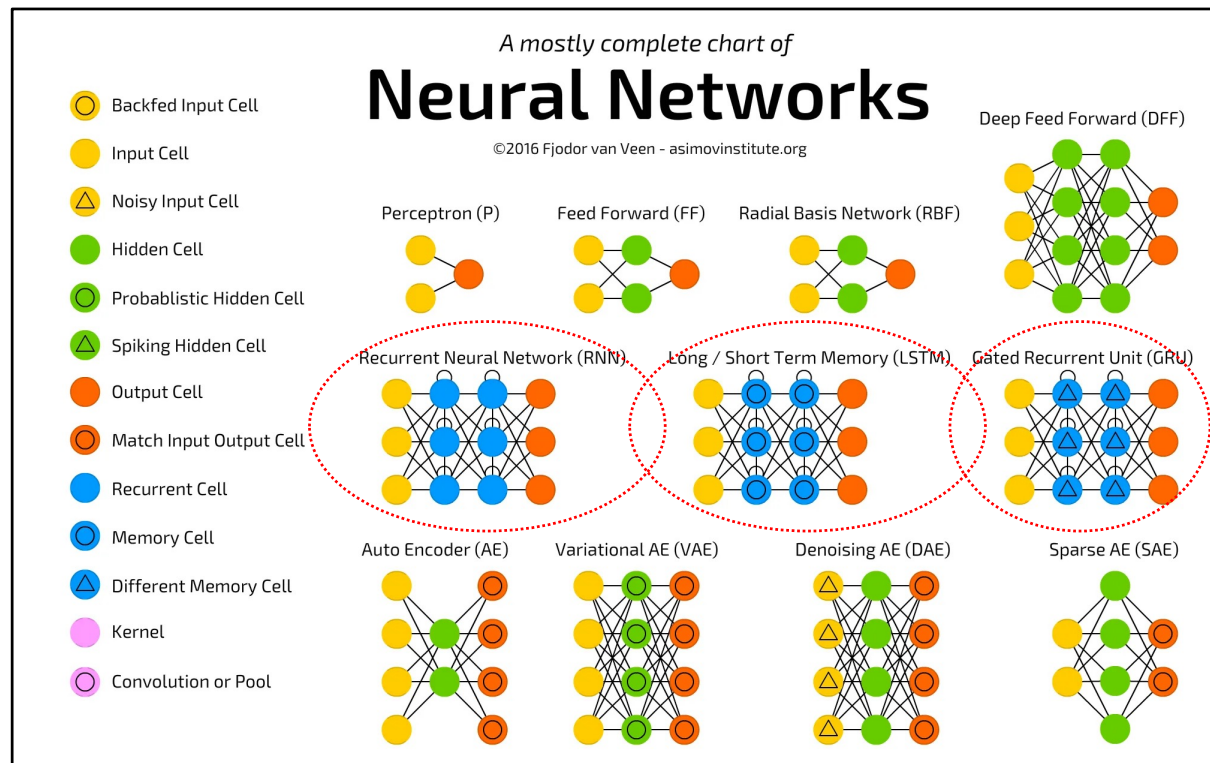
Fig. 1.17. A Fourier network

Figure 1.17 shows how a Fourier series can be implemented as a neural network. If the function F is to be developed as a Fourier series it has the form

$$F(x) = \sum_{i=0}^{\infty} (a_i \cos(ix) + b_i \sin(ix)). \quad (1.2)$$

Rojas, 1986

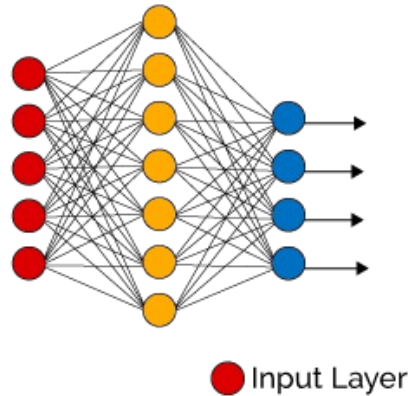
Types of neural networks



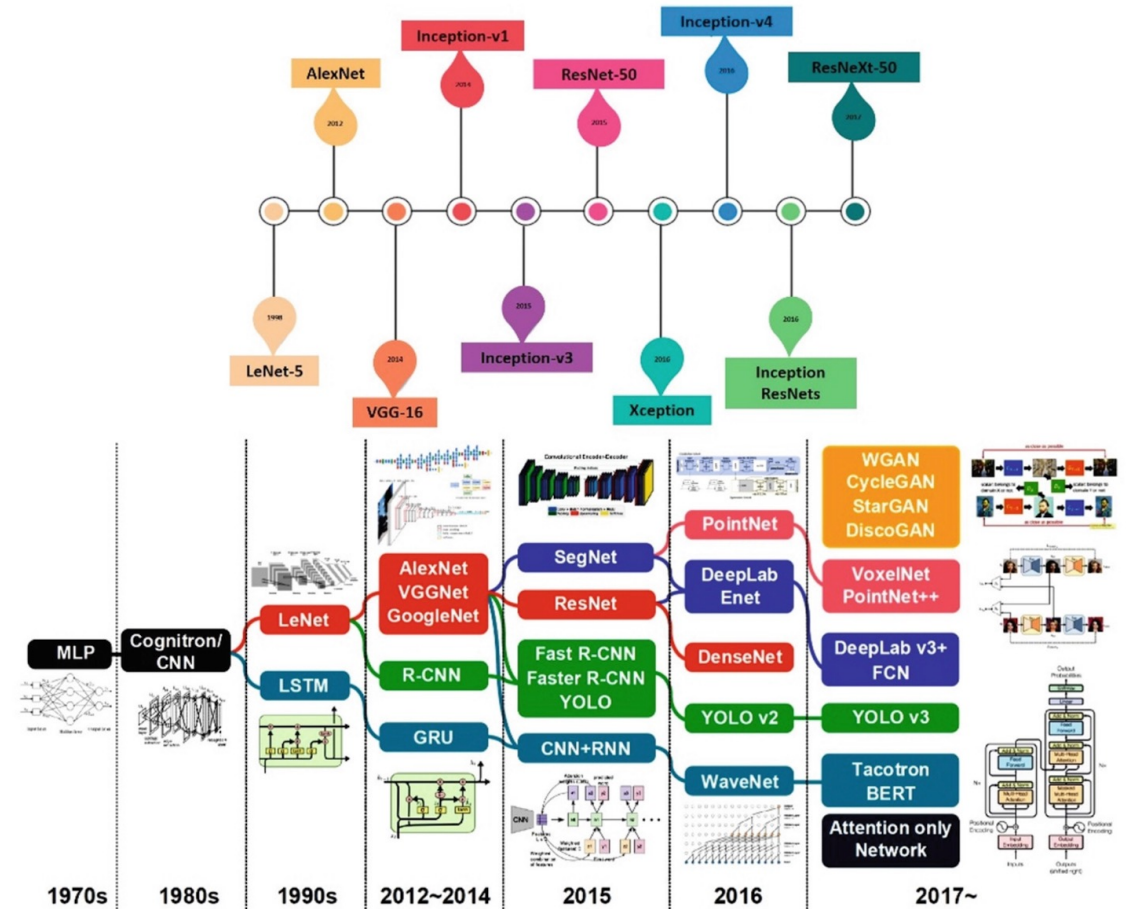
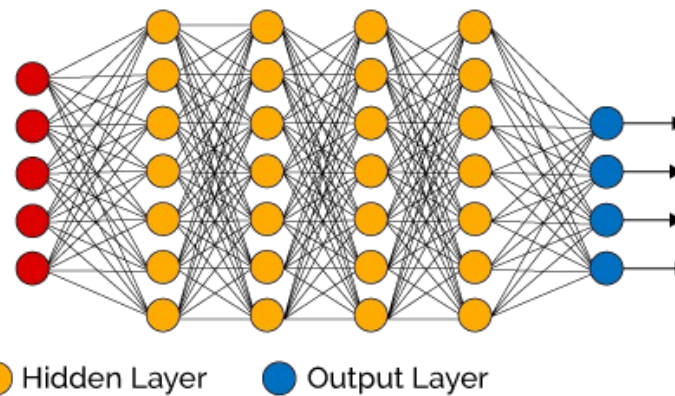
(Fjodor van Veen, 2016, asimovinstitute.org)

Deep neural networks

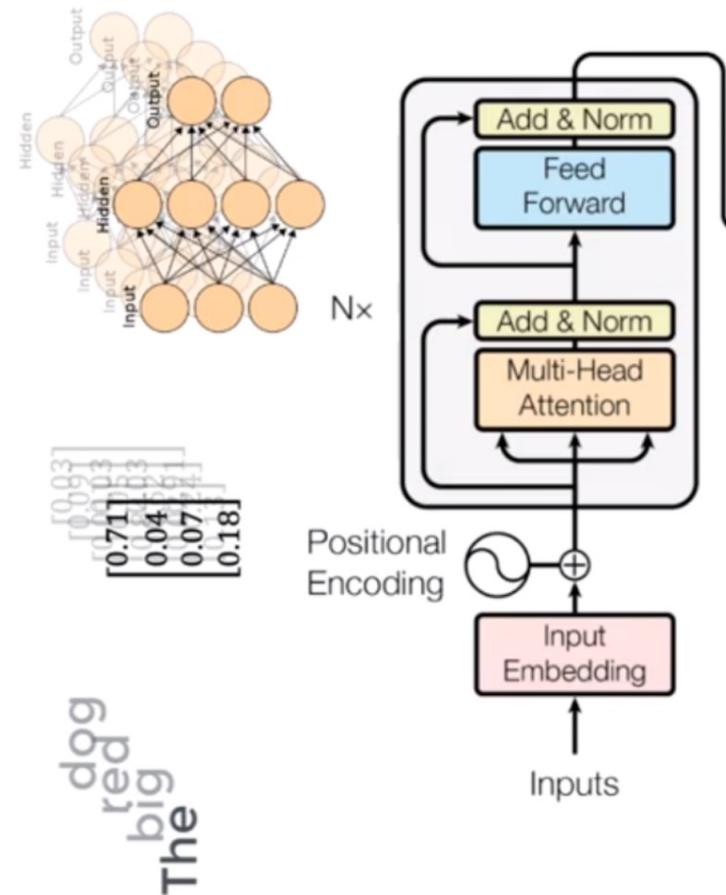
Simple Neural Network



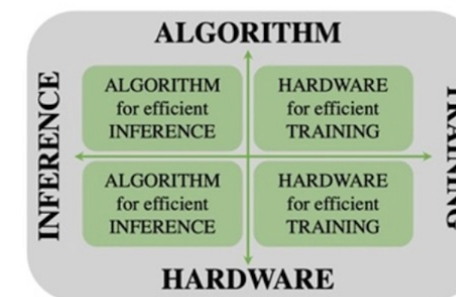
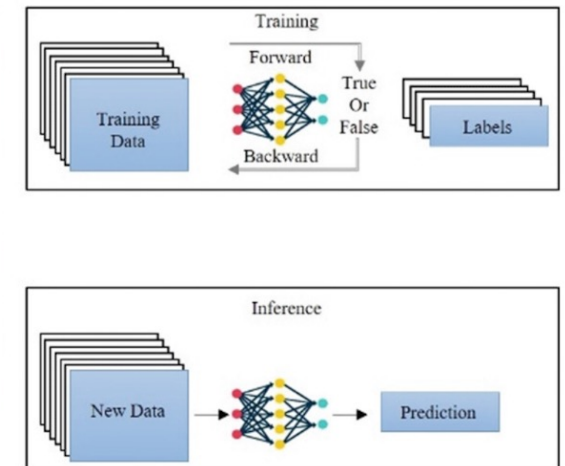
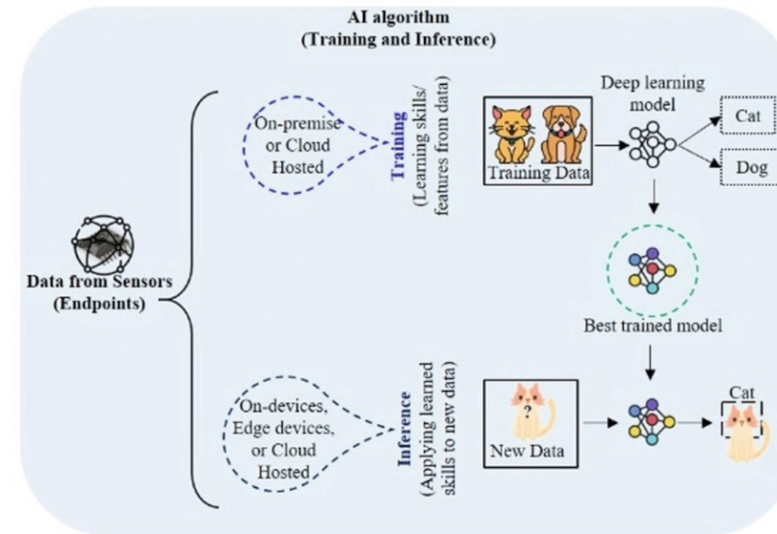
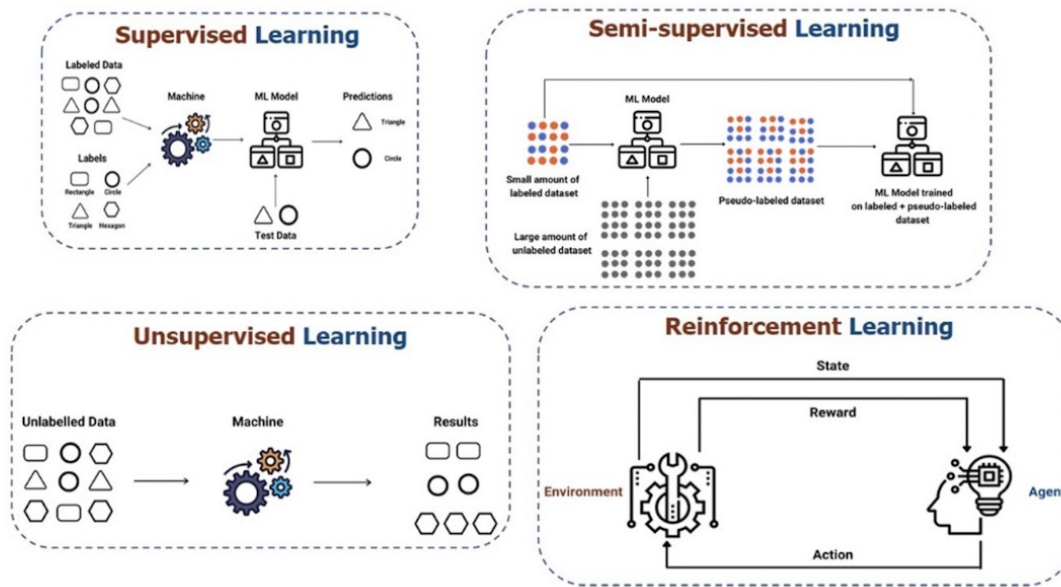
Deep Learning Neural Network



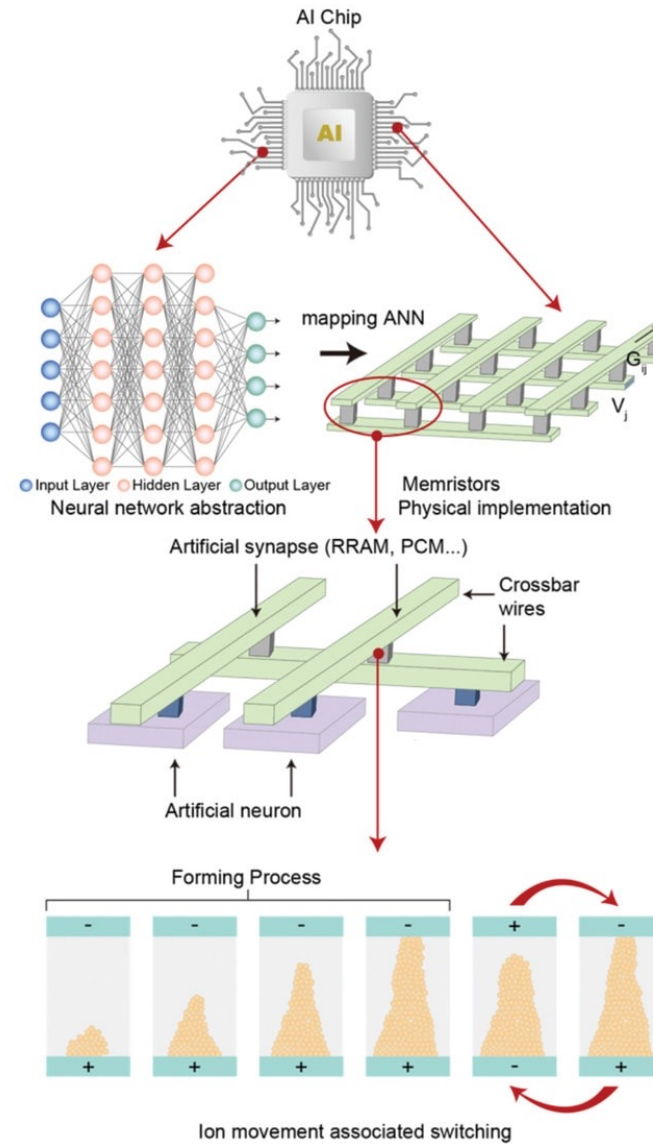
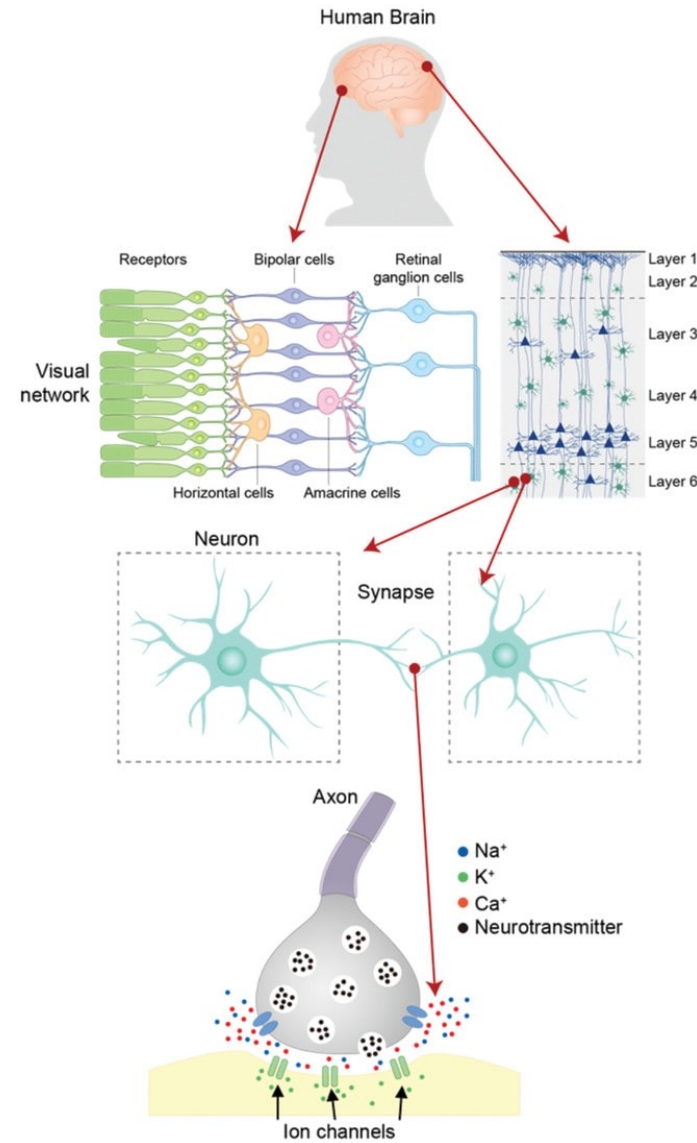
Transformers are deep neural networks



Learning



What is harder? Inference or training?





Neuromorphic Computing with Large Scale Spiking Neural Networks

Preprints.org (www.preprints.org) | NOT PEER-REVIEWED | Posted: 20 March 2025

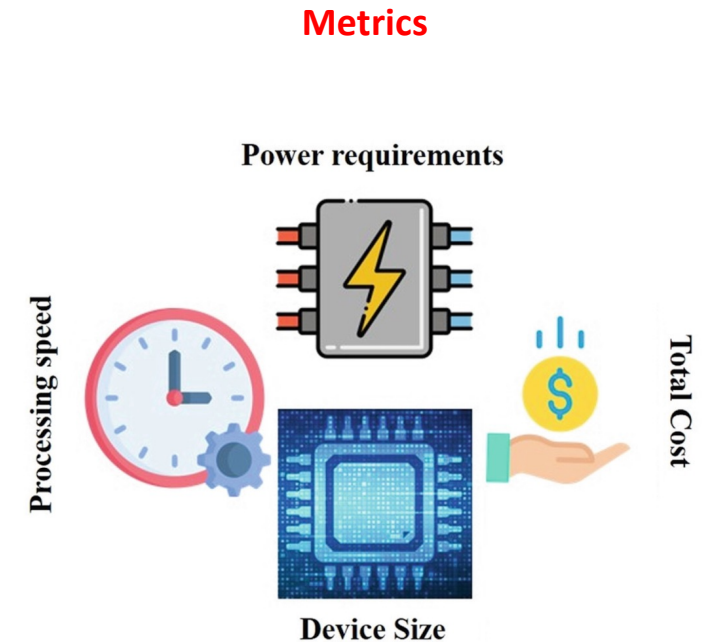
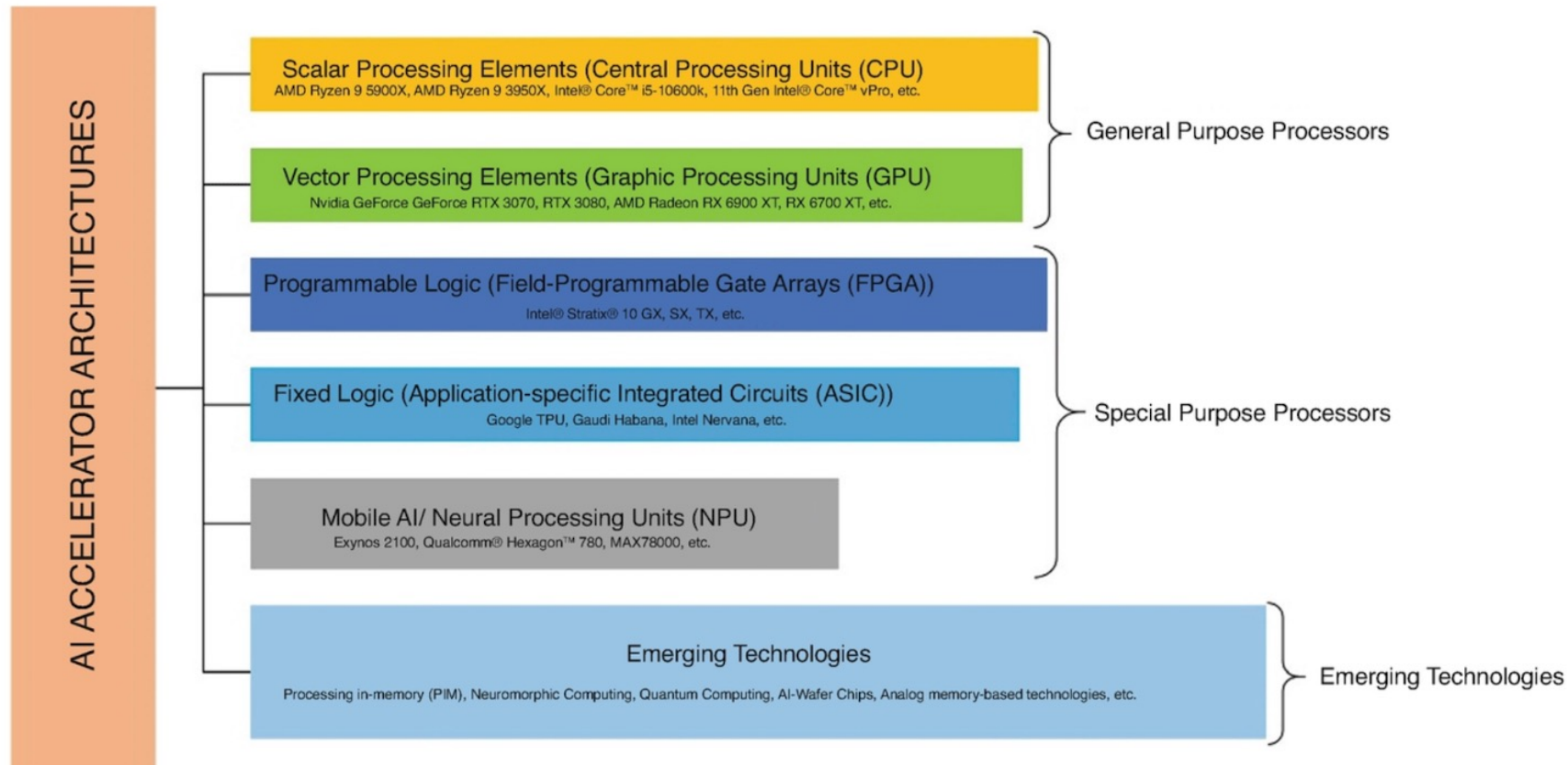
doi:10.20944/preprints202503.1505.v1

6 of 15

Table 3. Comparison of leading neuromorphic hardware platforms in terms of neuron capacity, power efficiency, and scalability.

Hardware Platform	Technology	Neuron Capacity	Power Efficiency	Scalability
IBM TrueNorth	Digital	10^6 neurons	High	Moderate
Intel Loihi	Digital	10^5 neurons	Very High	High
SpiNNaker	Digital	10^6 neurons	Moderate	High
BrainScaleS	Analog	10^4 neurons	Low	Low
Tianjic	Hybrid	10^5 neurons	High	High

Hardware for AI/ML



Hardware for AI/ML

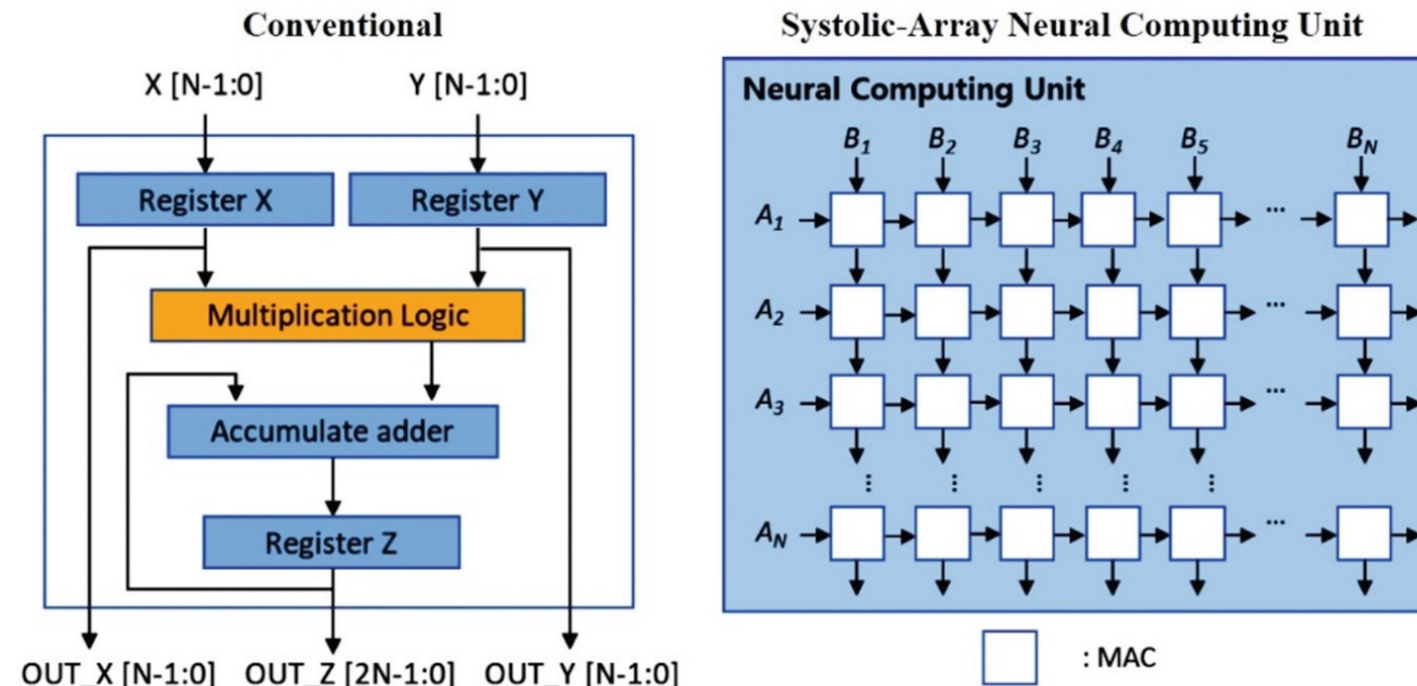


Fig. 19 A comparison of MAC operations performed on conventional and neural computing units.

Hardware for AI/ML

Processor	Power Consumption	Strengths	Limitations
CPU	High	→ Flexible → General-purpose processing → Complex instructions and tasks → System management	→ Possible memory access bottlenecks → Few cores (4–16)
GPU	High	→ Parallel cores (~1000 s of cores) → High-performance AI processing	→ Power consumption → Large footprint
FPGA	Medium	→ Configurable logic gates → Flexible → In-field re-programmability	→ Programming complexity
ASIC	Low	→ Custom logic designed with libraries → Faster processing → Small footprint	→ Fixed function → Expensive custom design
Vision Processing Unit (VPU)	Ultra-low	→ Dedicated image and vision co-processor → Small footprint	→ Limited dataset and batch size → Limited network support
Tensor Processing Unit (TPU)	Low to medium	→ Specialized tool support → Optimized for Tensor-Flow	→ Proprietary design → Limited framework support



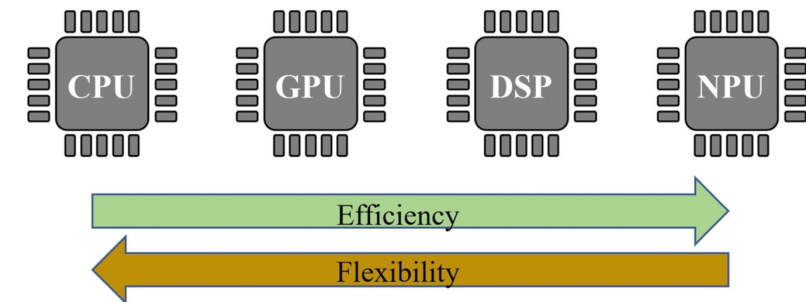
- Domain programmable
- Partially general-purpose
- Parallel workloads
- Power-hungry



- Hardware programmable
- Partially general-purpose
- Any workloads
- High performance
- Low power

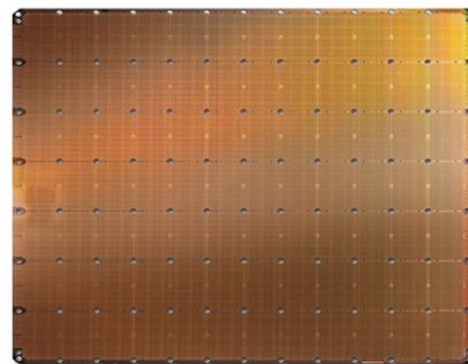
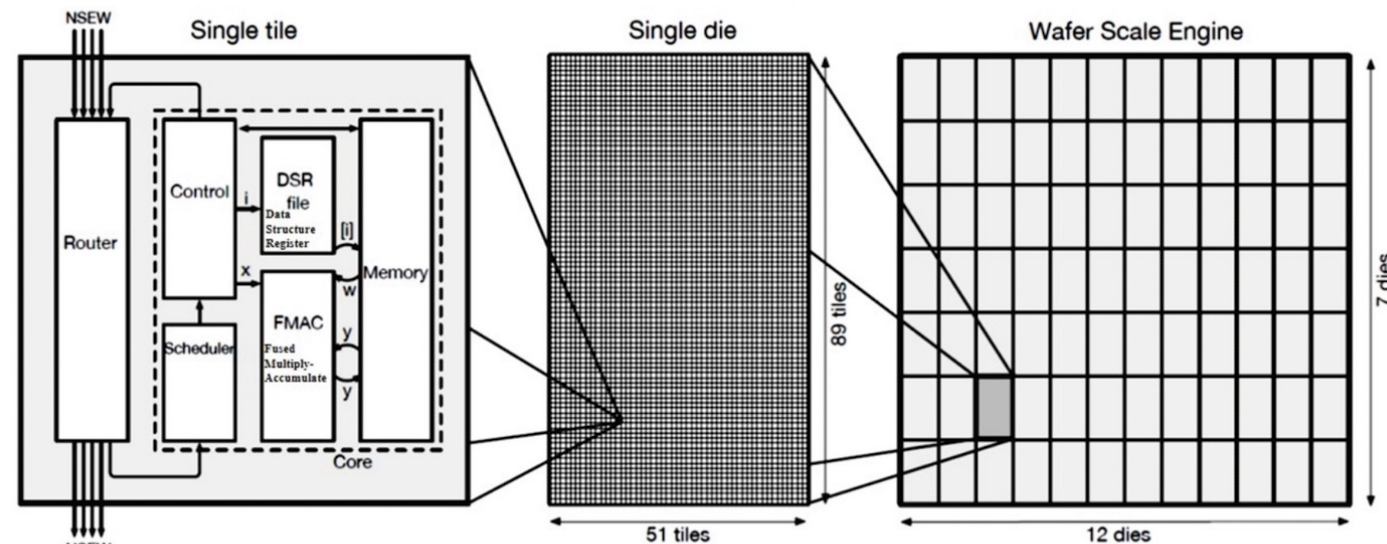


- Hard wired
- Special-purpose
- Dedicated workloads
- High performance
- Low power



Hardware for AI/ML

Cerebras Wafer Scale Engine (WSE)



	Gen1 WSE	Gen2 WSE
Fabrication process	16 nm	7 nm
Silicon area	46,225 mm ²	46,225 mm ²
Transistors	1.2 Trillion	2.6 Trillion
AI-optimized cores	400,000	850,000
Memory on-chip	18 GB	40 GB
Memory bandwidth	9 PB/s	20 PB/s
Fabric bandwidth	100 Pb/s	220 Pb/s

Mishra et al., 2023

Co-design & LLM



Calculon: a Methodology and Tool for High-Level Codesign of Systems and Large Language Models

Mikhail Isaev
michael.v.isaev@gatech.edu
Georgia Institute of Technology
Atlanta, Georgia, USA

Larry Dennison
ldennison@nvidia.com
NVIDIA
Westford, Massachusetts, USA

Nic McDonald
nimcdonald@nvidia.com
NVIDIA
Salt Lake City, Utah, USA

Richard Vuduc
richie@cc.gatech.edu
Georgia Institute of Technology
Atlanta, Georgia, USA

ABSTRACT

This paper presents a parameterized analytical performance model of transformer-based Large Language Models (LLMs) for guiding high-level algorithm-architecture codesign studies. This model derives from an extensive survey of performance optimizations that have been proposed for the training and inference of LLMs; the model's parameters capture application characteristics, the hardware system, and the space of implementation strategies. With such a model, we can systematically explore a joint space of hardware and software configurations to identify optimal system designs under given constraints, like the total amount of system memory. We implemented this model and methodology in a Python-based open-source tool called Calculon. Using it, we identified novel system designs that look significantly different from current inference and training systems, showing quantitatively the estimated potential to achieve higher efficiency, lower cost, and better scalability.

ACM Reference Format:
Mikhail Isaev, Nic McDonald, Larry Dennison, and Richard Vuduc. 2023. Calculon: a Methodology and Tool for High-Level Codesign of Systems and Large Language Models. In *The International Conference for High Performance Computing, Networking, Storage and Analysis (SC '23)*, November 12–17, 2023, Denver, CO, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3581784.3607102>

1 INTRODUCTION

We consider the task of conducting high-level analyses for algorithm-architecture codesign of distributed clusters and transformer-based Large Language Models (LLMs) [3, 4, 36, 39, 40]. By “high level,” we mean focusing on developing and using fast and coarse-grained analytical or semi-empirical models of the software and hardware, a stage of design that precedes detailed simulation or implementation on actual hardware. The goal is to estimate the best-case relative improvements that might come from significant changes to the system or software, as well as combinations of system and

This work is licensed under a Creative Commons Attribution International 4.0 License.
SC '23, November 12–17, 2023, Denver, CO, USA
© 2023 Copyright held by the owner/authors.
ACM ISBN 979-8-4007-4109-2/23/11.
<https://doi.org/10.1145/3581784.3607102>

Feature



A Survey: Collaborative Hardware and Software Design in the Era of Large Language Models

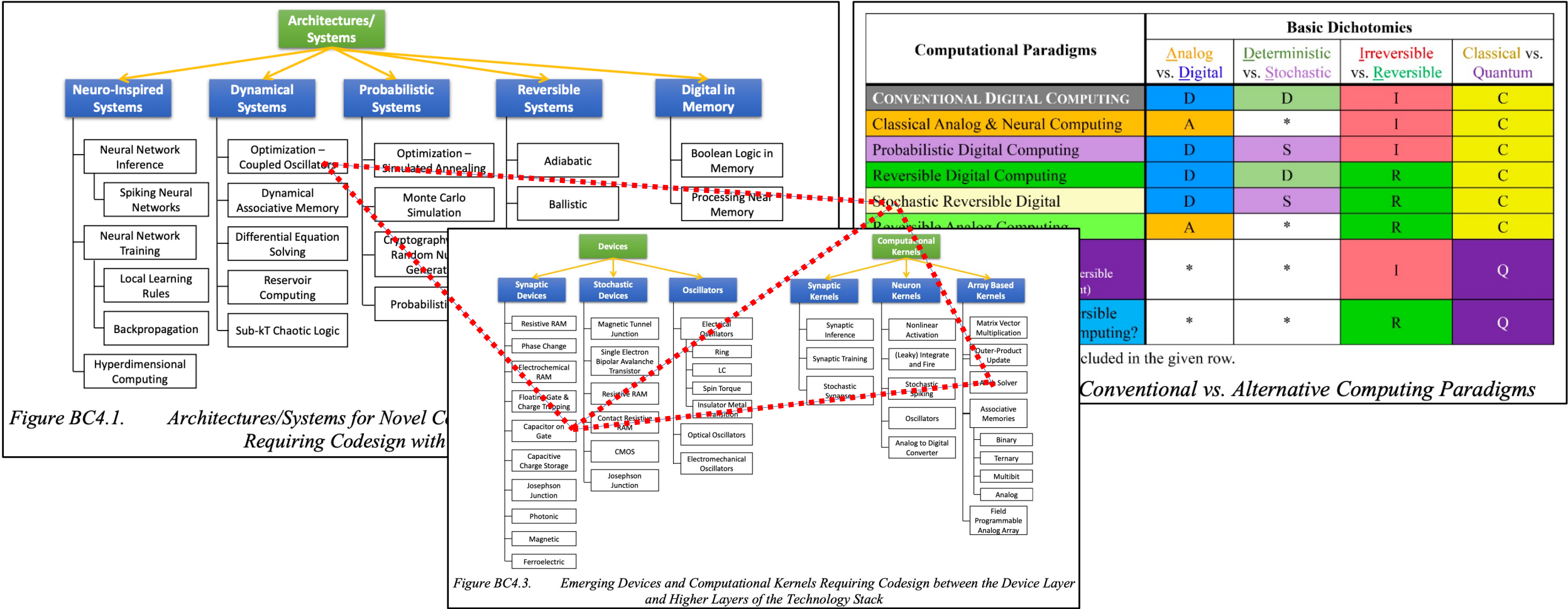
Cong Guo, Feng Cheng, Zhixu Du, James Kiessling, Jonathan Ku, Shiyu Li, Ziru Li, Mingyuan Ma, Tergel Molom-Ochir, Benjamin Morris, Haoxuan Shan, Jingwei Sun, Yitu Wang, Chiyue Wei, Xueying Wu, Yuhao Wu, Hao Frank Yang, Jingyang Zhang, Junyao Zhang, Qilin Zheng, Guanglei Zhou, Hai Li, *Fellow, IEEE*, and Yiran Chen, *Fellow, IEEE*

Digital Object Identifier 10.1109/MCAS.2024.3476008
Date of current version: 7 February 2025

(HW/SW) Co-design

- **Classical approach:** in college, different technologies are taught in different classes.
- **Hardware-software co-design:**
 - Started in the 1990s.
 - Its core idea is the concurrent designs of hardware and software components of complex electronic systems.
- **More broadly:**
 - The concurrent design of hardware and software across all layers of the compute stack.
- **What are possible metrics and trade-offs?**

Deep co-design across the entire stack



Food for thought

- Why is executing a neural network on a general purpose computer inefficient?
- What circuitry/processor would you design to find the shortest path in large graphs?
 - What if the graph is dense?
 - What if the graph is sparse?
- What circuitry/processor would you design to recognize QR codes for an embedded camera?
 - Recognition needs to be fast and low-power.